

1. مقدمة المشروع

يهدف هذا المشروع إلى بناء خط أنابيب بيانات لحظي (Real-Time Data Pipeline) لمعالجة بيانات يتم توليدها بشكل مستمر، ثم تمر عبر مراحل متعددة تشمل الجمع، المعالجة، النقل، والتخزين النهائي باستخدام تقنيات Big Data.

2. توليد البيانات باستخدام Python

في البداية تم إنشاء سكربت بلغة Python يقوم بتوليد بيانات بشكل مستمر لمحاكاة بيانات تطبيقات حقيقية.

وظيفة السكربت:

- إنشاء ملفات CSV بشكل دوري
- محاكاة بيانات Logs خاصة بتطبيقات الجوال

خصائص البيانات:

تم تصميم البيانات لتكون واقعية و غير نظيفة وتشمل:

- قيم مفقودة (Missing Values)
- بيانات مكررة (Duplicate Records)
- قيم غير صحيحة (Invalid Data)
- تواريخ بصيغ مختلفة
- قيم رقمية سالبة أو غير منطقية
- صفوف تالفة (Corrupted Rows)

الهدف:

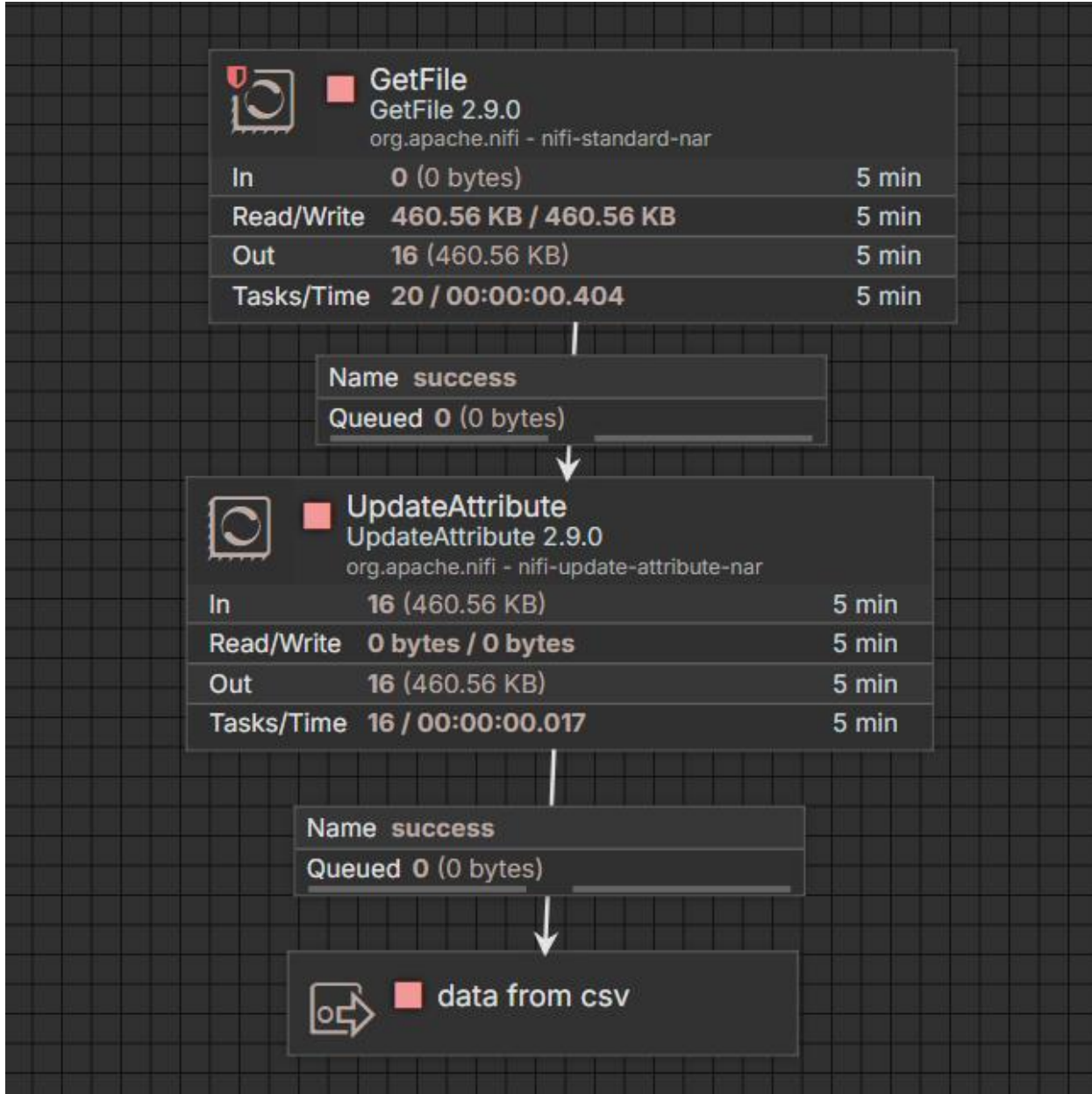
محاكاة بيئة بيانات حقيقية غير منظمة لاختبار قدرات المعالجة.

3. استقبال البيانات باستخدام Apache NiFi

بعد توليد الملفات، تم استخدام Apache NiFi لمراقبة مجلد الإدخال واستقبال الملفات تلقائيًا.

وظيفة NiFi:

- قراءة ملفات CSV بشكل مستمر
- إدارة تدفق البيانات (FlowFiles)



4. تحويل البيانات (ConvertRecord Processor)

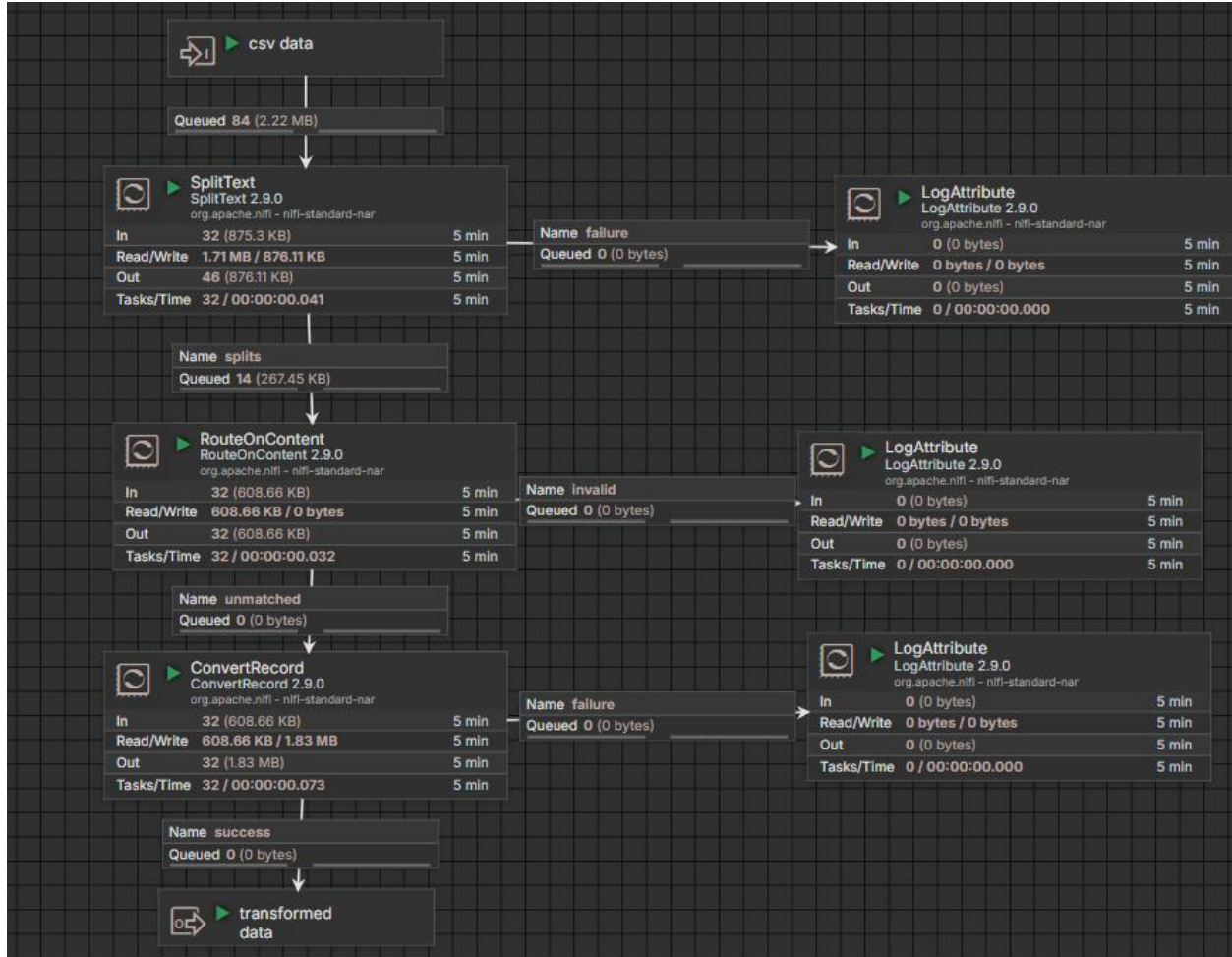
تم استخدام Processor باسم ConvertRecord داخل NiFi.

الوظيفة:

- تحويل البيانات من CSV إلى JSON
- استخدام:
 - CSVReader لقراءة البيانات
 - JsonRecordSetWriter لإخراج البيانات بصيغة JSON

الهدف:

تحويل البيانات إلى صيغة منظمة قابلة للإرسال والتخزين.



5. إرسال البيانات إلى Apache Kafka.

بعد معالجة البيانات، يتم إرسالها إلى Apache Kafka.

إعدادات:Kafka

- إنشاء Topic باسم logs :
- إرسال البيانات من NiFi إلى Kafka باستخدام PublishKafka Processor

الوظيفة:

- تمكين تدفق البيانات بشكل لحظي (Streaming)
- فصل طبقة المعالجة عن طبقة التخزين

```
[appuser@b4504e589bd0 ~]$ kafka-topics --create \  
> --topic logs \  
> --bootstrap-server localhost:9092 \  
> --partitions 3 \  
> --replication-factor 1  
Created topic logs.  
[appuser@b4504e589bd0 ~]$ |
```

وهذه البيانات التي ادخلتها الى كافكا

```
[appuser@b4504e589bd0 ~]$ kafka-console-consumer \  
> --bootstrap-server localhost:9092 \  
> --topic logs \  
> --from-beginning  
  
[ {  
  "log_id" : 8856,  
  "device_id" : "SM-A52",  
  "log_level" : "DEBUG",  
  "event_name" : "button_click",  
  "response_time_ms" : 157,  
  "timestamp" : "2026-05-15 05:36:16"  
}, {  
  "log_id" : 8610,  
  "device_id" : "IPHONE-13",  
  "log_level" : "DEBUG",  
  "event_name" : "api_call",  
  "response_time_ms" : 999999999,  
  "timestamp" : "05-15-2026 05:36:16 AM"  
}, {  
  "log_id" : 8306
```

6. تخزين البيانات في Hadoop HDFS.

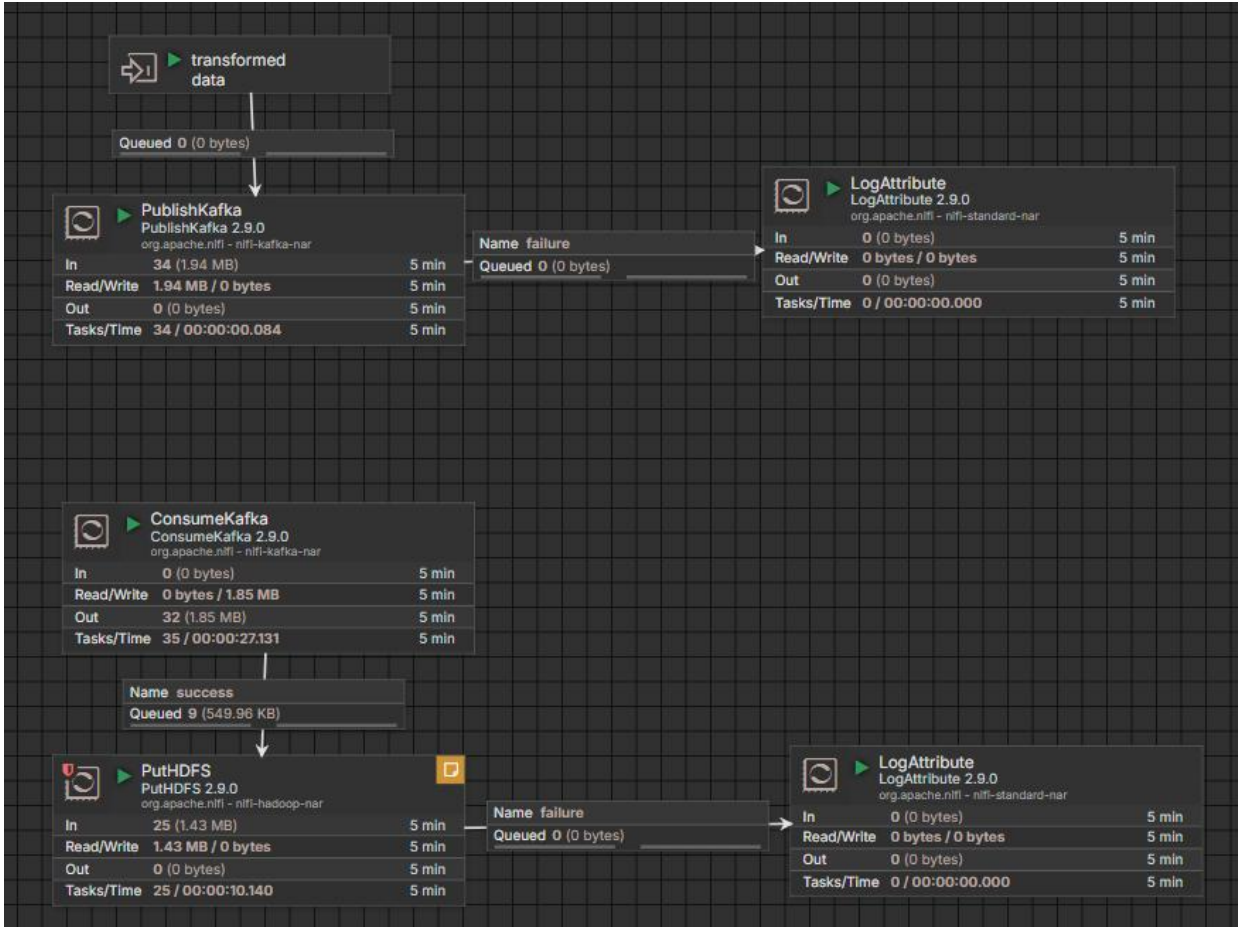
بعد تمرير البيانات عبر Kafka ، يتم تخزينها في HDFS باستخدام NiFi.

المسار المستخدم:

/data/logs/year=YYYY/month=MM/day=DD

الهدف:

- تنظيم البيانات بشكل زمني (Partitioning)
- تسهيل عمليات التحليل المستقبلية



7. مشاكل تم مواجهتها أثناء التنفيذ

خلال بناء المشروع تم مواجهة عدة مشاكل تقنية:

1. مشاكل في عنوان HDFS (fs.defaultFS) بين localhost واسم الحاوية
2. مشاكل صلاحيات في HDFS (Permission Denied)
3. أخطاء في Kafka مثل عدم تطابق Cluster ID
4. مشاكل في الاتصال بين الحاويات داخل Docker network
5. أخطاء في ConvertRecord بسبب عدم تطابق schema

8. الحلول التي تم تطبيقها

تم حل المشاكل السابقة من خلال:

- إنشاء مجلد مخصص داخل HDFS /data/logs
- تعديل الصلاحيات للسماح بالكتابة
- استخدام أسماء الحاويات داخل Docker بدلاً من localhost
- إعادة ضبط Kafka وحذف volume عند الحاجة
- تعديل إعدادات NiFi وربطها بشكل صحيح مع HDFS
- التأكد من صحة CSV schema قبل التحويل

9. تقسيم البيانات (Partitioning Strategy)

تم اعتماد تقسيم البيانات داخل HDFS حسب التاريخ بالشكل التالي:

```
year=YYYY
└─ month=MM
    └─ day=DD
```

الفائدة:

- تحسين أداء الاستعلامات
- تنظيم البيانات بشكل منطقي
- تسهيل التحليل الزمني

10. خصائص الاعتمادية في النظام

تم تصميم النظام ليكون مستمرًا ومرنًا من خلال:

- توليد بيانات بشكل مستمر باستخدام Python
- إدارة التدفق باستخدام NiFi
- معالجة الأخطاء وإعادة المحاولة
- استخدام Kafka كطبقة نقل بيانات مستقلة
- تخزين البيانات بشكل آمن في HDFS